

Universidad Tecnológica Nacional – Facultad Regional Rosario
Ingeniería en Sistemas de Información
Cátedra de Redes de Información

CAPA DE RED
CARACTERÍSTICAS DE TRANSPORTE

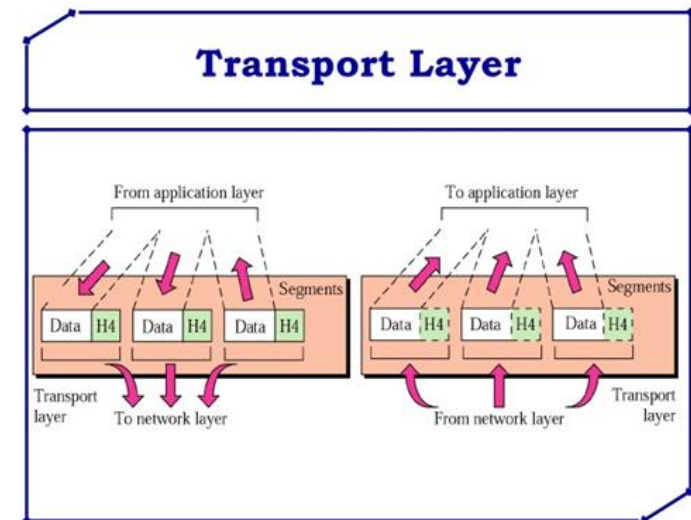
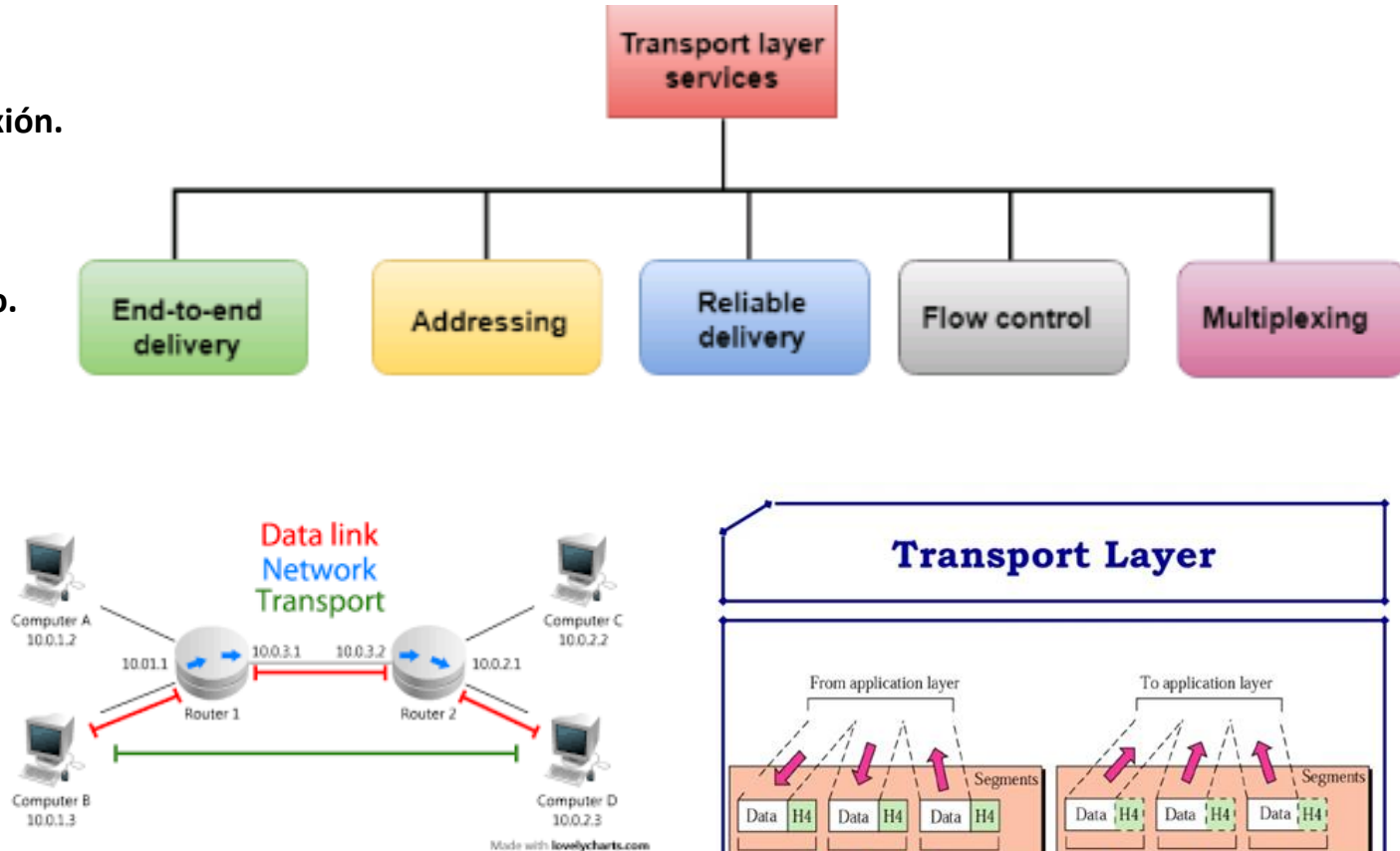
Ing. Hernán Vitri
Ing. Germán Baró
Ing. Esteban Travaglino

CAPA 4 – LA CAPA DE TRANSPORTE

CARACTERÍSTICAS DE LOS PROTOCOLOS DE TRANSPORTE

- ÍNDICE:

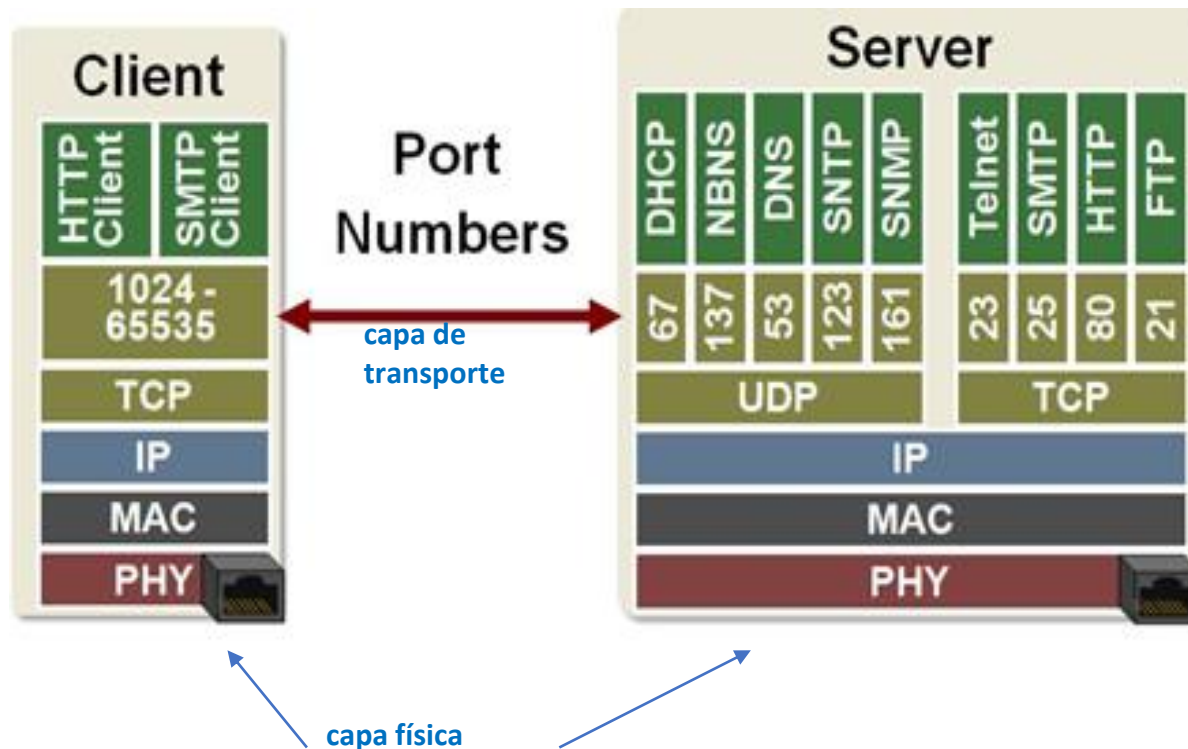
- 1- Direccionamiento.
- 2- Establecimiento de la conexión.
- 3- Desconexión.
- 4- Control de errores y de flujo.
- 5- Multiplexación.
- 6- Control de congestión.
- 7- Crash recovery.
- 8- Cuestiones inalámbricas.
- 9- Bibliografía.



1 – DIRECCIONAMIENTO.

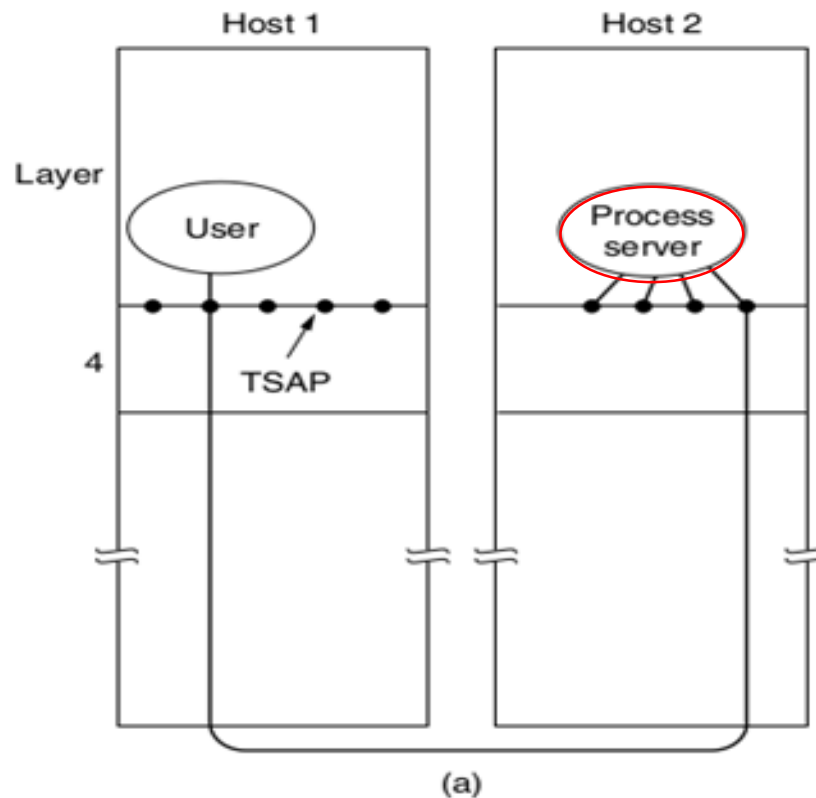
- **Puertos:** En capa 4 utilizamos la sigla **TSAP** (Transport Service Access Point) para indicar un extremo de conexión, que comúnmente denominamos “PUERTO”. Son 16 bits que brindan 65536 direcciones de transporte. Por lo general, un host utiliza varios puertos en capa 4 a la vez y todos con una sola dirección IP de capa 3. Muchas aplicaciones utilizan puertos conocidos por defecto: Telnet = 23, SMTP = 25, HTTP = 80, FTP = 21, etc.

Arquitectura cliente-servidor.



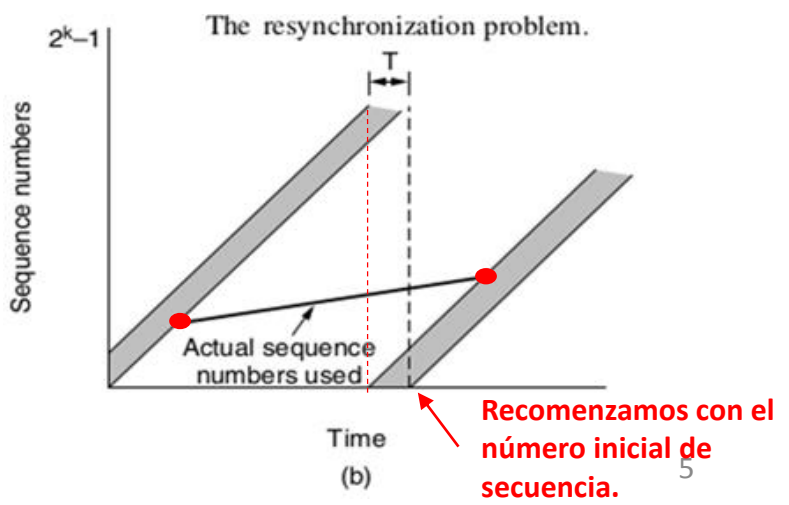
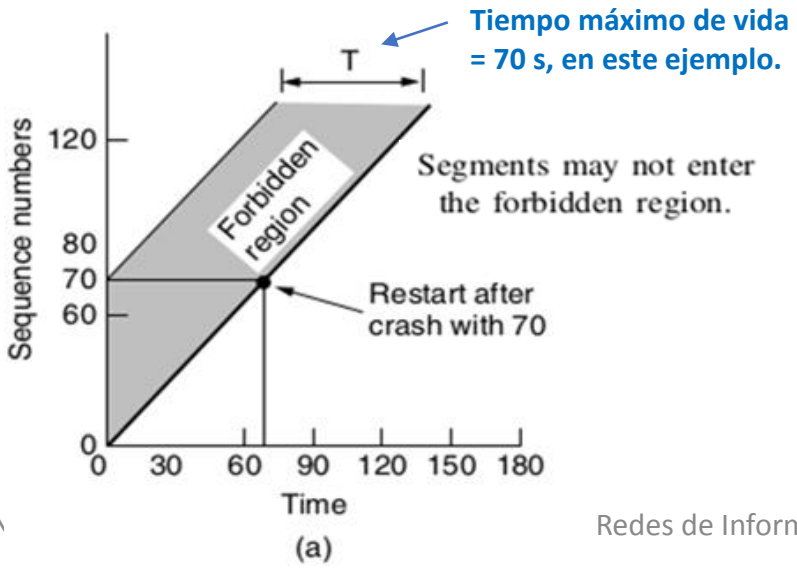
1 - DIRECCIONAMIENTO (continuación).

- Si el puerto destino es desconocido, hay un par de procesos alternativos para averiguarlo:
- **“Port mapper”**: Es un proceso que asocia un nombre de servicio a un número de puerto. El “Port mapper” utiliza por defecto el puerto conocido 111 y los hosts lo puede consultar. Le preguntan qué puerto utiliza un determinado servicio y el “port mapper” le responde con el número de puerto correspondiente para ese servicio.
 - **“Server Process”**: El servidor emplea un proceso llamado “server process” que monitorea varios puertos a la vez, como un “proxy”. Cuando un cliente busca ese servidor con un determinado puerto (TSAP), el “server process” aprovecha ese mismo puerto para conectarlo al servicio en cuestión.



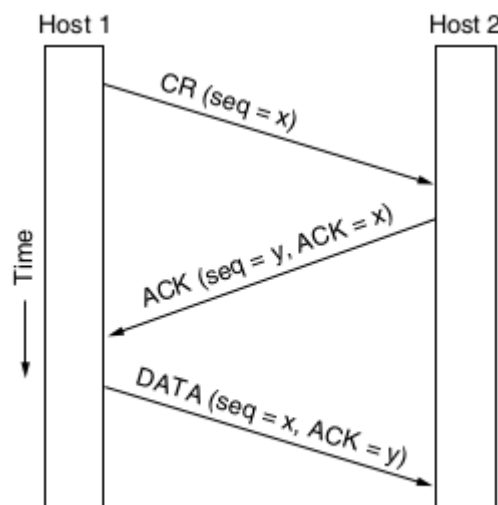
2 – ESTABLECIMIENTO DE LA CONEXIÓN.

- **Complicaciones:** En servicios orientados a la conexión (TCP), para establecer una conexión en Capa 4, una entidad de transporte envía un segmento **“Connection Request”** a una entidad remota y queda esperando un segmento de **“aceptación”**. Aunque parezca sencillo entablar una conexión, cuando en una red se presentan anomalías, aparecen complicaciones. Evitar la duplicación de segmentos es crítico.
- ✓ **Ejemplo:** Imaginemos un cliente solicitando a un banco que transfiera una buena cantidad de dinero a otra persona. Supongamos que los paquetes toman una ruta más larga y demoran varios segundos en llegar al banco. Por lo tanto, el cliente podría considerar que los paquetes se han extraviado y reenvía el pedido de transferencia. El banco finalmente recibe ambos pedidos y realiza dos veces la transferencia de dinero, pensando que son transferencias independientes.
- El problema anterior puede resolverse de varias maneras. La mejor forma es limitando la vida de un segmento por tiempo o por saltos, y limitando también el tiempo de vida de los **“acknowledgements”** del receptor. Para evitar segmentos duplicados, se establece un **“tiempo máximo de vida”**, T, de alrededor de 120 segundos (valor utilizado en la internet actual). Entonces se debe esperar un mínimo tiempo T para repetir el segmento con el mismo número de secuencia, sabiendo que los viejos paquetes con esa secuencia ya fueron descartados de la red.

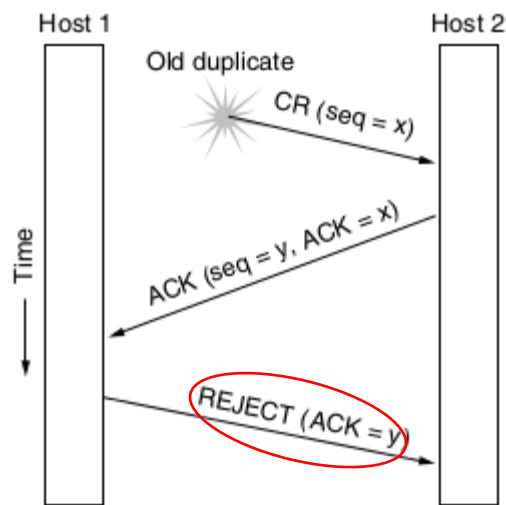


2 – ESTABLECIMIENTO DE LA CONEXIÓN (continuación).

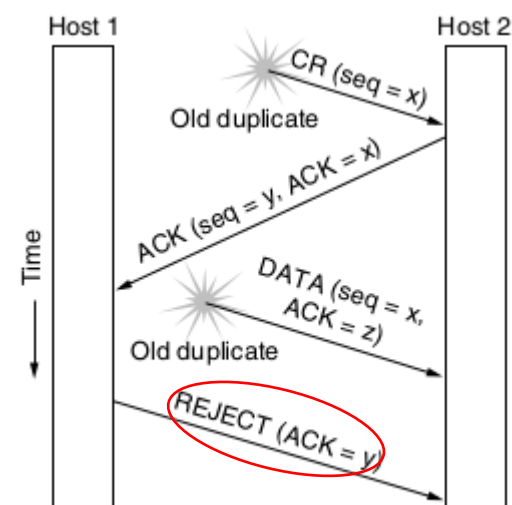
- **Método de las tres vías, (Three-way handshake):** En la práctica se emplea el método de Tomlinson (1975), o método de las 3 vías, luego refinado por Sunshine and Dalal (1978). Consiste en agregar un número de secuencia en cada segmento, que no puede ser reusado en cierto tiempo T , para garantizar que los segmentos “gemelos” se hayan extinguido. El número de secuencia se extrae de un reloj en tiempo real, que cada host tendrá, y no es necesario que estén sincronizados. El host 2 contesta con un “acknowledgement” de la secuencia usada por host 1, y luego el host 1 responde ya utilizando su secuencia con un “acknowledgement” de la secuencia elegida por host 2.
- **TCP** usa este método para establecer conexiones, con 32 bits de secuencia y un valor inicial aleatorio para evitar predicciones por algún presunto atacante. (RFC 1323, PAWS = Protection Against Wrapped Sequence Number).



(a)
Normal operation.



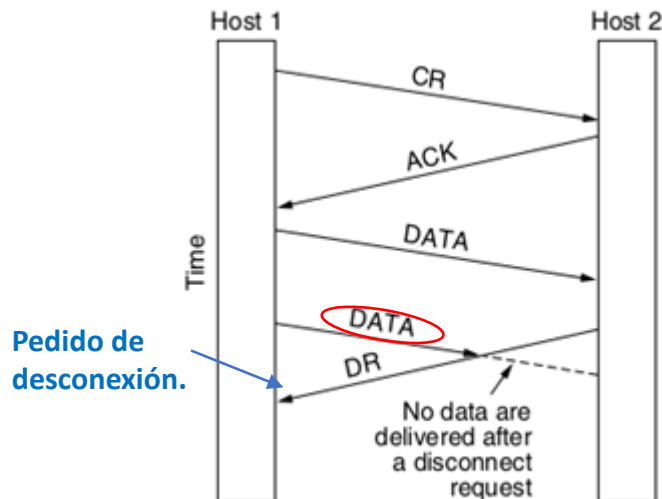
(b)
Old duplicate CONNECTION REQUEST
appearing out of nowhere.



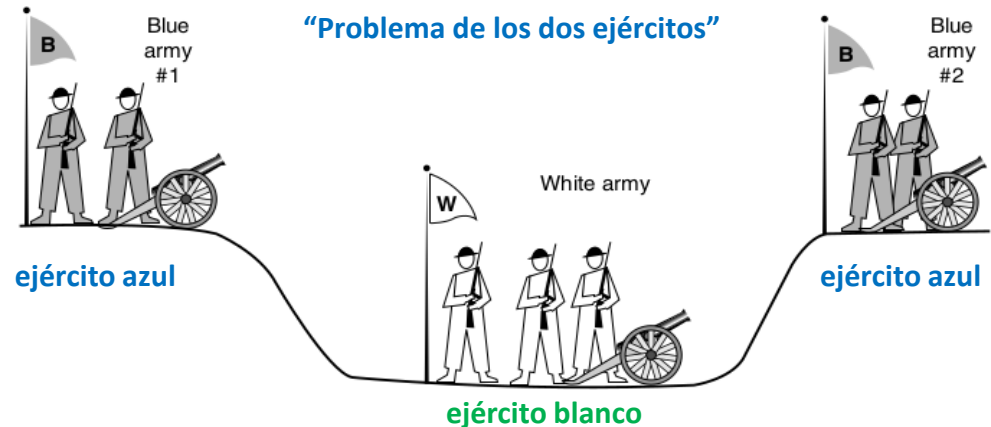
(c)
Duplicate CONNECTION REQUEST
and duplicate ACK.

3 – DESCONEXIÓN.

- **Desconexión simétrica:** La desconexión es coordinada de ambos lados. En general, la desconexión es más sencilla que la conexión. Sin embargo, hay algunos obstáculos que sortear. Habría que evitar la desconexión abrupta de un solo lado, como se observa en la figura, ya que podría ocasionar pérdidas de segmentos. La desconexión puede ser simétrica, o asimétrica. Es simétrica cuando se interpreta la comunicación como dos canales unidireccionales que tienen que desconectarse en forma independiente. Es asimétrica, por ejemplo, en una llamada telefónica, donde si alguno de los extremos cuelga, finaliza la comunicación.
- **Problema de los dos ejércitos:** Si la desconexión es simétrica, existe un problema comparable al “problema de los dos ejércitos”, que se muestra en la figura. Si el ejército azul ataca en conjunto, vence al ejército blanco. Pero si ataca por separado, el blanco lo supera. El dilema es cómo coordinar las acciones del ejército azul 1 y 2, porque tendrán que enviar mensajeros, que al pasar por el valle podrían ser capturados por el ejército blanco. Supongamos que el ejército azul 1 envía su mensaje y llega con éxito al ejército azul 2. Sin embargo, el ejército 1 no atacará porque no está seguro si el mensaje llegó efectivamente. Supongamos que el mensaje del ejército 1 llega bien, y el ejército 2 envía la confirmación de que recibió el mensaje al ejército 1. Sin embargo, ahora quién no sabe si atacar o no, es el ejército 2, ya que no sabe si su mensaje ha sido recibido por el ejército 1.



Abrupt disconnection with loss of data.



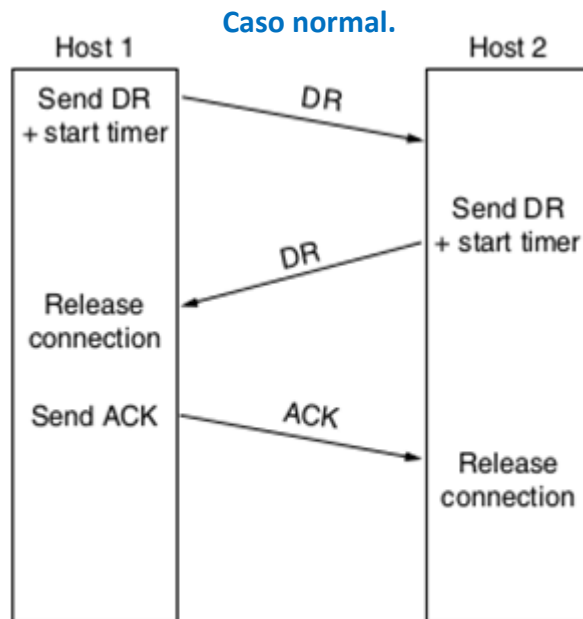
The two-army problem.

3 - DESCONEXIÓN (continuación)

- **Desconexión en 3 vías, o “handshake” de 3 vías:** Esta sucesión de mensajes no tiene solución, ya que el último mensaje nunca se sabe si fue recibido por el otro ejército. Esto mismo sucede en la desconexión. No se sabe si el último mensaje fue recibido o no. En la práctica se deja que cada lado decida independientemente cuando desconectar, y por lo tanto, el protocolo para la desconexión coordinada es también un “handshake de 3 vías”.
- Veamos 4 ejemplos distintos de desconexión:

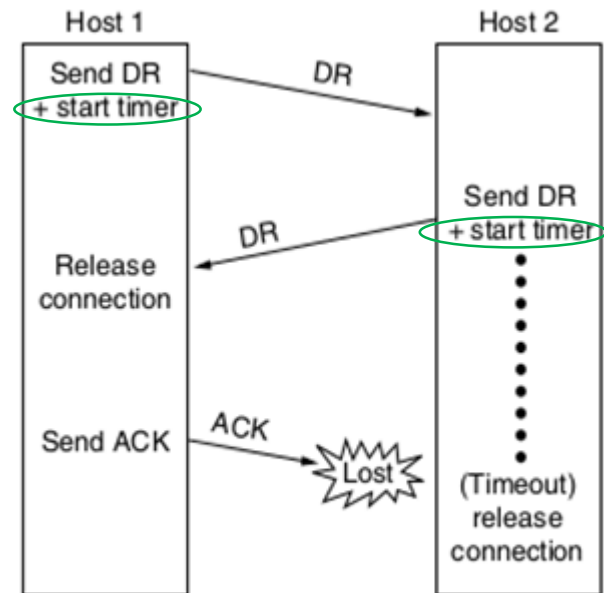
➤ 1- En figura (a), el proceso es el deseado.

2- En figura (b), se pierde un “acknowledgement”. En este caso, después de un cierto tiempo (timeout) se procede igual a la desconexión.



(a)

Normal case of three-way handshake.

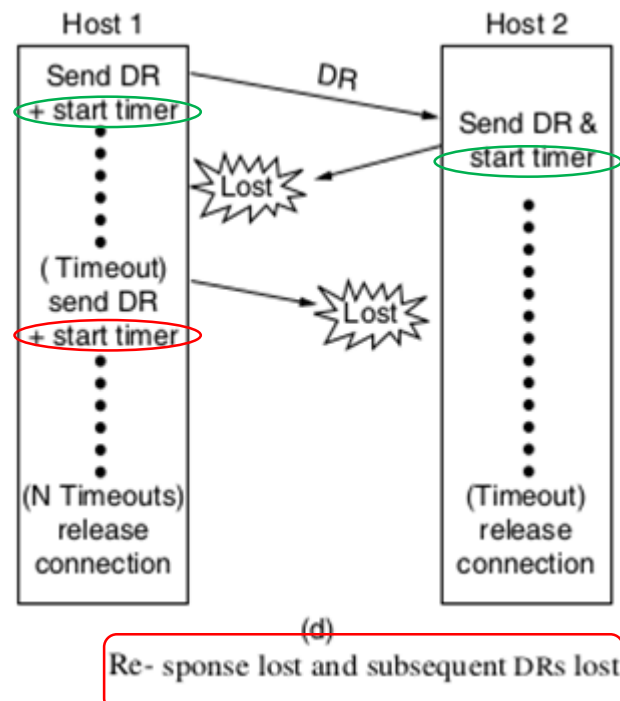
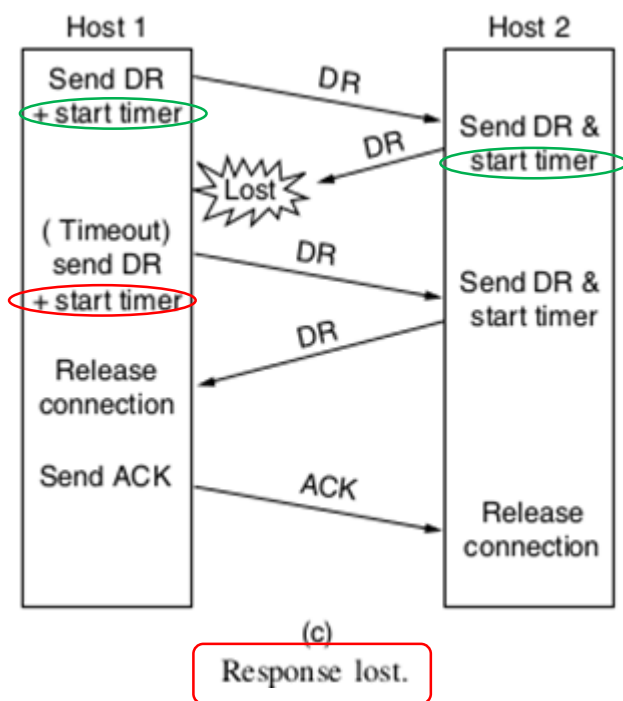


(b)

Final ACK lost.

3 - DESCONEXIÓN (continuación).

- 3- En figura (c) se pierde el “Disconnection Request” del host 2. Entonces el host 1 vuelve a enviar el mensaje de “Disconnection Request”
- 4- En la última figura (d), se presenta el caso más complicado. Se pierde el “Disconnection Request” del host 2 y también las N réplicas del host 1. La desconexión se realiza igual al excederse los tiempos (timeout) establecidos tanto para host 1, como en host 2.
- En caso de que se pierdan todos los segmentos enviados por host 1, la desconexión debe llevarse a cabo igual, después de un tiempo prudencial. El receptor, al no recibir segmentos por cierto tiempo, también debe finalizar la conexión. Este sería el caso, por ejemplo, si el enlace quedara físicamente fuera de servicio.



4 – CONTROL DE ERRORES Y DE FLUJO.

- En capa 4 se emplean los mismos mecanismos de control de errores y de flujo que en capa 2, aunque con distintos grados y funciones. No olvidemos que la capa de enlace trabaja salto a salto, mientras que la capa de transporte opera de extremo a extremo.
- **Control de errores:** Al final de cada segmento hay un campo “CRC o checksum” para detectar errores. En TCP es obligatorio, mientras que en UDP es opcional, como luego veremos.
- **Control de flujo:** La ventana deslizante establece un número de secuencia para identificar el mensaje y el “acknowledgement” correspondiente. Si no se recibe el “acuse de recibo”, se vuelve a reenviar el segmento después de un cierto tiempo hasta que el receptor confirme su recepción. Este proceso se conoce como **ARQ** (Automatic Repeat reQuest).

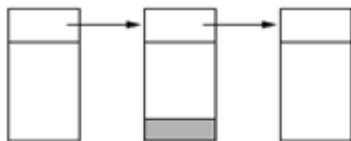
Observaciones:

- ✓ **Router:** Si se producen errores dentro de un router, la capa de enlace no los detectará y sí, en cambio, serán detectados por la capa de transporte.
- ✓ **WiFi:** En cuanto a la ventana deslizante, muchos protocolos inalámbricos, como el estándar WiFi, sólo transmiten una trama y esperan su confirmación, (stop and wait). Una ventana deslizante más grande aumentaría la complejidad sin mejorar la performance. Esto sucede cuando el producto velocidad por retardo es bajo.
- ✓ **Ethernet:** En enlaces cableados, como Ethernet, no se controlan los errores porque es baja la tasa de error y se deja que las retransmisiones extremo a extremo de Capa 4 reparen los errores.
- ✓ En algunas transmisiones por TCP, que tienen un bajo producto Retardo por Ancho de Banda, se emplea una ventana deslizante mayor para lograr mayor eficiencia. Por ejemplo, en un enlace de 1 Mbps pero con 2 ms de retardo ida y vuelta. Esto quiere decir que se almacenarían 0,2 Mbits en el búfer del receptor hasta que se reciba un “acknowledgment” (2 ms). Por eso en este caso se emplea una ventana deslizante mayor.

4 – CONTROL DE ERRORES Y DE FLUJO (continuación).

- **Buffers:** Por lo general, los protocolos de transporte utilizan “ventanas deslizantes” grandes, y por lo tanto, analizaremos el problema de los “buffers” de almacenamiento. Cuando arriba un nuevo segmento, se asigna un nuevo buffer. El mismo tamaño de buffer es lo más sencillo de implementar, aunque también podemos definir tamaños diferentes como en la figura (b), o un espacio circular como en figura (c), que tal vez sea la mejor solución.
- Inicialmente, el emisor solicita un cierto número y tamaño de cada buffer, según su velocidad y cantidad de datos, y el receptor informa la disponibilidad. Así, con cada “acknowledgement”, el receptor informa sobre el número de segmento recibido y el espacio de memoria aún disponible.

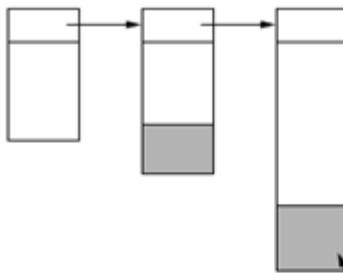
mismo tamaño de buffer



Chained fixed-size buffers.

(a)

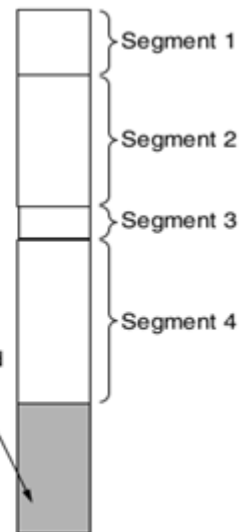
distinto tamaño de buffer



Chained variable-sized buffers.

(b)

buffer circular



(c)

One large circular buffer per connection.

4 – CONTROL DE ERRORES Y DE FLUJO (continuación).

- Ejemplo: En la figura se observa una comunicación entre A y B, donde se detallan las confirmaciones del receptor respecto de los segmentos recibidos y el espacio de memoria.
- **Ventana deslizante dinámica**: Cuando el espacio de memoria no es el problema, aparece otro cuello de botella que es la capacidad de la red. Y si el emisor envía más tráfico que el que se puede transmitir, se produce una congestión. Por lo tanto, una ventana deslizante dinámica ajusta el flujo a la capacidad de la red y al espacio disponible en memoria.

	A	Message	B	Comments
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	...	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has 1 buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	...	<ack = 6, buf = 4>	←	Potential deadlock

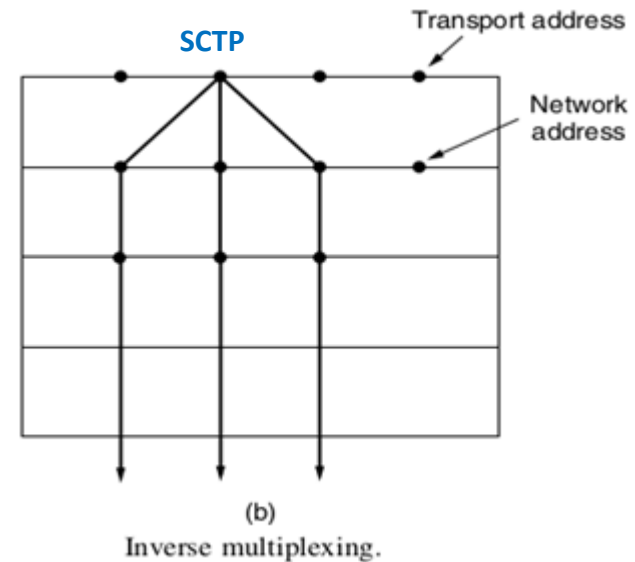
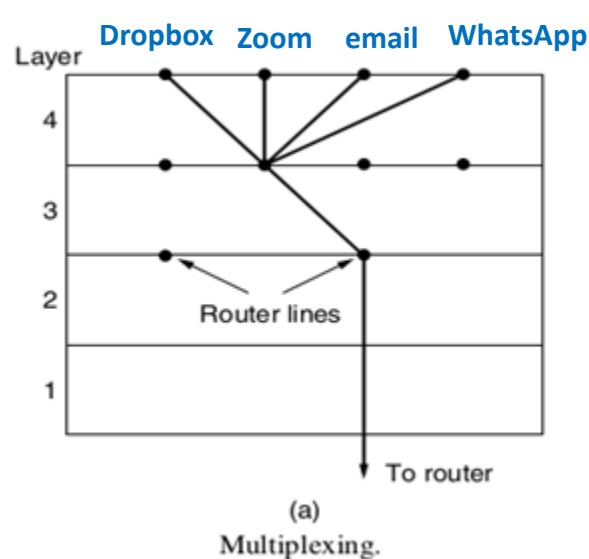
Las flechas indican la dirección del mensaje.
Los puntos suspensivos indican que se ha perdido el mensaje.

B se queda sin lugar en el buffer.

B encuentra 4 nuevos buffers pero el mensaje se pierde. A queda bloqueado sin poder transmitir. Para prevenir esto, cada host debe enviar periódicamente mensajes de control informando acerca del estado de los buffers en cada conexión. De este modo, más tarde o más temprano, este falso bloqueo de A se interrumpe.

5 – MULTIPLEXACIÓN.

- En el caso que un host tenga una única dirección de red, todas las conexiones de capa 4 deben compartir ese mismo enlace en el host. En la figura (a) vemos un ejemplo de multiplexación, mientras que en la figura (b) se observa un ejemplo de multiplexación inversa.
- Un ejemplo de multiplexación inversa es SCTP, (Stream Control Transmission Protocol). Para lograr una velocidad efectiva mayor en el receptor, este protocolo utiliza en paralelo varias conexiones de red disponibles. También hay casos de multiplexación inversa en Capa 2, para mejorar la velocidad de transmisión con múltiples enlaces, por ejemplo en WiFi.
- En **TCP** no se realizan procesos de multiplexación. TCP es unicast, (un emisor y un receptor).

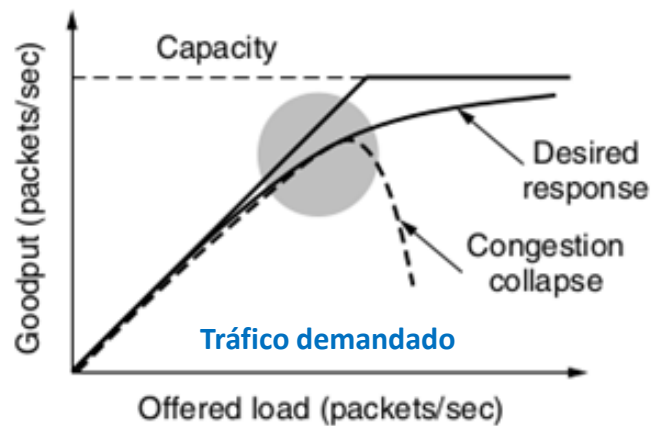


6 – CONTROL DE CONGESTIÓN.

- El manejo de una congestión es responsabilidad conjunta de las capas de red y de transporte. La congestión ocurre en un router y por lo tanto es detectada en la capa de red, pero es causada por el tráfico enviado desde la capa de transporte. El objetivo del control de congestión es asignar una tasa de bits adecuada a cada entidad de transporte que comparta la red, de modo de lograr la mejor performance posible sin llegar a congestionar la red.
- Tráfico vs retardo:** La figura (a) muestra el “goodput”, o sea, el tráfico cursado por la red en función del tráfico demandado. Se observa en gris la zona de congestión. Y la figura (b) indica el retardo, (delay) como señal de congestión. El mínimo retardo está dado por el tiempo de propagación a través de la red.
- En 1979 Kleinrock propuso la métrica siguiente para representar la performance de la capa 4:

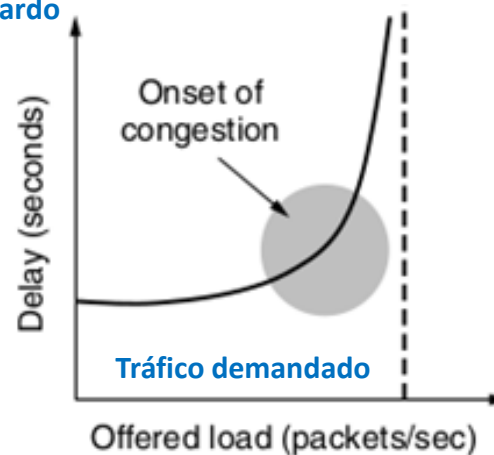
$$\text{Performance} = \text{Tráfico/Retardo}$$

Tráfico cursado.



(a)
Goodput as a function of offered load.

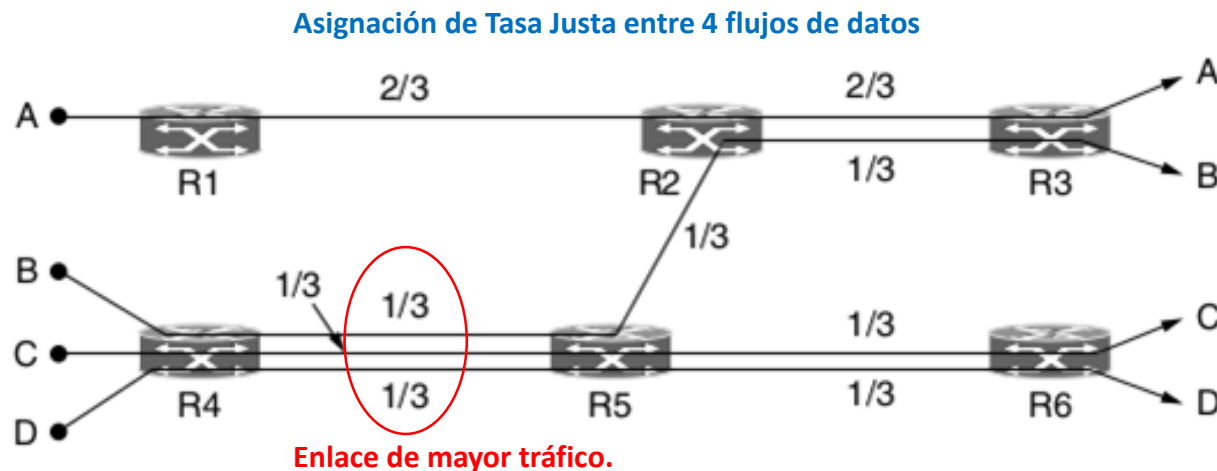
Retardo



(b)
Delay as a function of offered load.

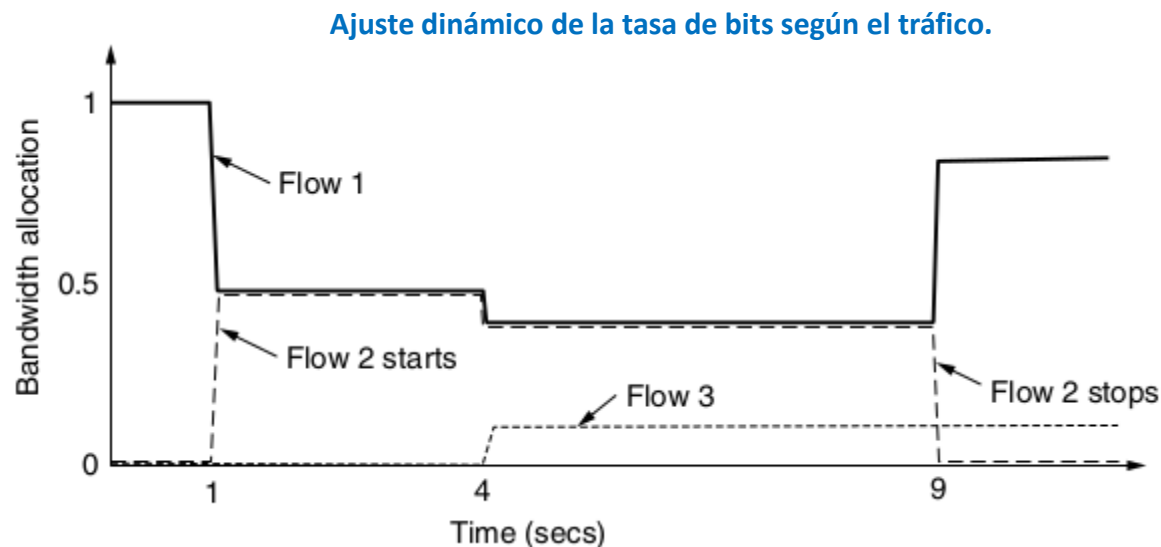
6 – CONTROL DE CONGESTIÓN (continuación).

- **Tasa Justa:** Es común que las redes no reserven tasa de bits para las distintas fuentes y que cada una utilice el máximo disponible. Por lo tanto, es el mecanismo de control de congestión quien limita la tasa de bits de cada fuente de transmisión. Decimos que la tasa de bits asignada a una fuente es “Máx-Mín Fair” (Tasa Justa) si cualquier aumento repercute en una disminución de la tasa de bits de otra fuente.
- Ejemplo: En la figura se observa una asignación Máx-Mín-Justa entre 4 fuentes, A, B, C y D. Se supone que todos los enlaces tienen la misma capacidad = 1. El enlace de abajo izquierda es el de mayor tráfico y por lo tanto la asignación para las fuentes B, C y D debe ser $1/3$. Entonces, la fuente restante, A, puede disponer de un ancho de banda de $2/3$.



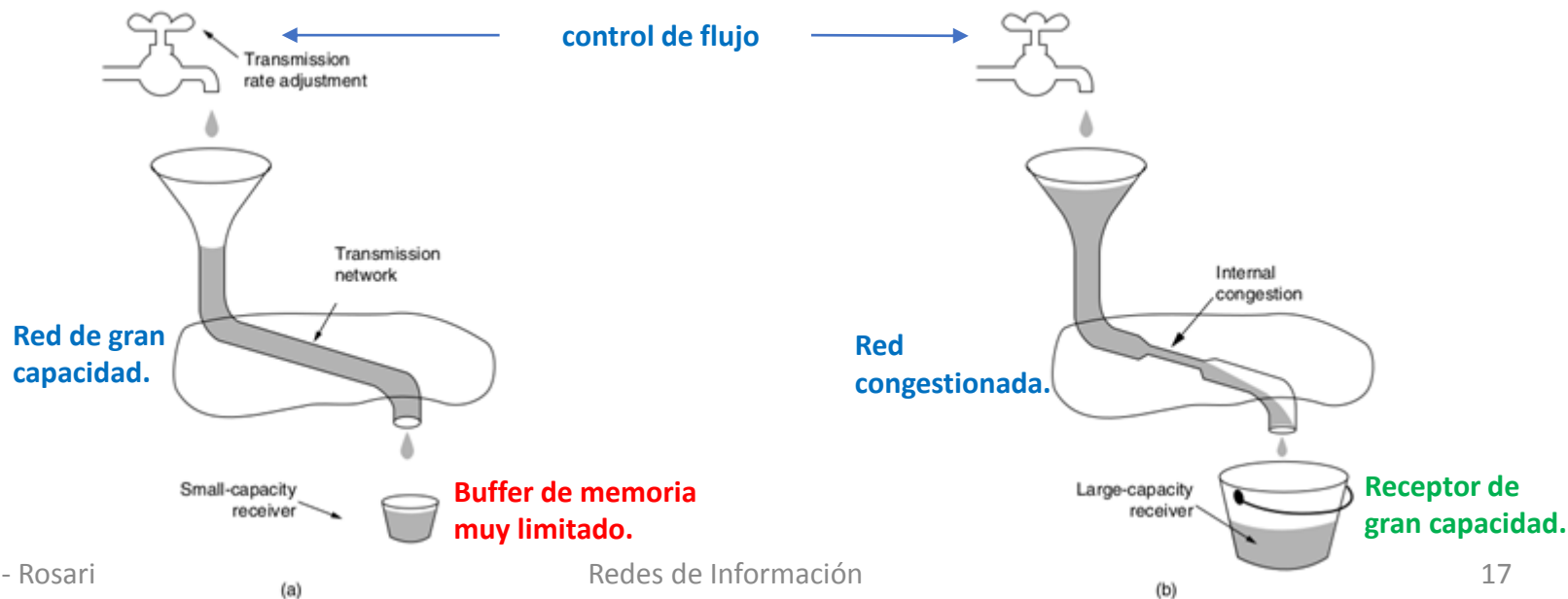
6 – CONTROL DE CONGESTIÓN (continuación).

- El algoritmo de asignación de tasa de bits debe converger rápidamente a una asignación de Tasa Justa.
- Comúnmente las fuentes de tráfico no son constantes, sino que dinámicamente abren y cierran conexiones. Por lo tanto, la asignación correcta de tasa de bits debe ser dinámica y ajustarse periódicamente, como se muestra en la figura.
- Una forma intuitiva de lograr una asignación de Tasa Justa es que cada fuente comience desde cero a incrementar su tráfico hasta que aparezca una congestión en algún tramo y allí en ese punto no aumente más. Aunque algunas aplicaciones abren varias conexiones a la vez, para obtener de la red un mayor ancho de banda efectivo, por ejemplo, “BitTorrent”.



6 – CONTROL DE CONGESTIÓN (continuación).

- **¿Cómo regular el tráfico de cada fuente?** Observemos la analogía hidráulica de las figuras:
 - En figura (a) encontramos una red de gran capacidad conectada a un receptor con buffer de memoria muy limitado. En este caso debe existir un buen mecanismo de control de flujo para que no se pierdan paquetes.
 - En figura (b) en cambio, tenemos una red congestionada entregando datos a un receptor de gran capacidad. Aquí debe existir una adecuada asignación de la tasa de bits entre las distintas fuentes.
- Pero en ambos casos debemos controlar adecuadamente el flujo de la fuente para no perder datos por provocar una congestión.
- Una forma es que cada router le explice a cada fuente el **flujo** permitido a transmitir.
 - Otra forma es que la Capa 4 notifique a las fuentes sobre los **retardos** de cada camino para evitar puntos de congestión.
 - En la práctica, la **pérdida** de paquetes también se utiliza como indicador de congestión.



6 – CONTROL DE CONGESTIÓN (continuación).

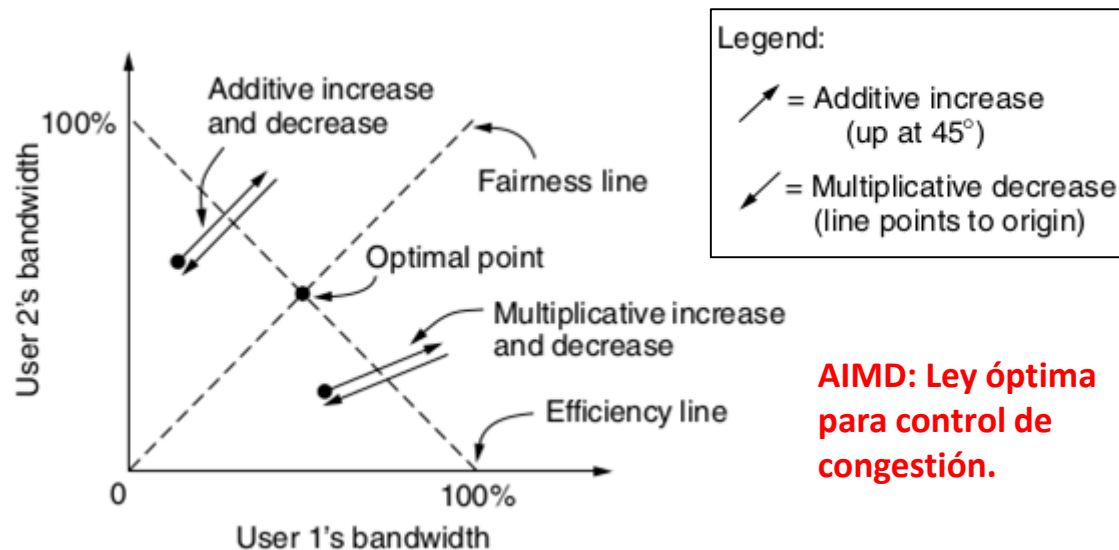
- Existen variantes de estos mecanismos de control de flujos en TCP, que se visualizan en la tabla de abajo:
 - **Linux:** utiliza un protocolo llamado “CUBIC TCP” que toma en cuenta las pérdidas de paquetes.
 - **Windows:** usa “Compound TCP”, que toma en cuenta ambas cosas, tanto el retardo como la pérdida de paquetes.
- **“FAST TCP”** es otro protocolo de control de congestión que mide el tiempo de propagación como métrica para evitar congestiones.

Protocol	Signal	Explicit?	Precise?
XCP	Rate to use	Yes	Yes
TCP with ECN	Congestion warning	Yes	No
FAST TCP	End-to-end delay	No	Yes
Windows → Compound TCP	Packet loss & end-to-end delay	No	Yes
Linux → CUBIC TCP	Packet loss	No	No
TCP	Packet loss	No	No

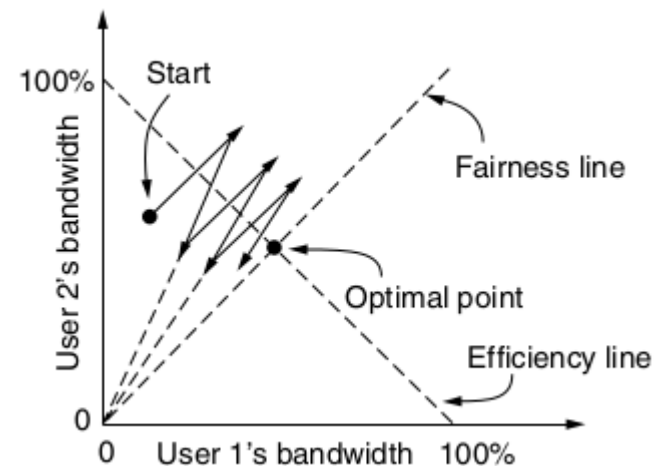
Figure 6-23. Signals of some congestion control protocols.

6 – CONTROL DE CONGESTIÓN (continuación).

- **AIMD, (Additive Increase Multiplicative Decrease):** Es la ley de control de congestión óptima, creada en 1989 por Chiu y Jain. Su funcionamiento se grafica en las figuras de abajo. Una variación aditiva en la tasa de bits se visualiza como una recta a 45 grados. Una variación multiplicativa se ve como una recta pasante por el origen.
- AIMD incrementa aditivamente y decrementa multiplicativamente, haciendo que la fuente con mayor tráfico disminuya más velozmente su flujo logrando que el mecanismo converja al punto de operación óptimo. Es la ley de control utilizada en TCP. De todas formas, no es del todo justo ya que TCP ajusta sus ventanas deslizantes según el “round trip time”, o tiempo de propagación extremo a extremo, haciendo que las conexiones más cortas tengan más tasa de bits que las conexiones más largas.
- Los protocolos de Capa 4 distintos a TCP, deben ser “TCP amigables” y usar una ley de control de congestión similar a TCP, ya que de lo contrario podrían apropiarse de la mayor parte de la tasa de bits compitiendo deslealmente contra el conocido y popular TCP.



**AIMD: Ley óptima
para control de
congestión.**



7 – CRASH RECOVERY.

- **Falla en host receptor:** Si eventualmente se interrumpe una comunicación por fallas en un router o en un enlace, se producen retransmisiones de los segmentos perdidos. Si en cambio, la falla se genera en el **host receptor**, resolver qué hacer es un poco más complicado.
- **Emisor:** Al enterarse de un “crash” en el receptor, el emisor tiene 4 alternativas posibles:
 - ✓ Retransmitir siempre.
 - ✓ Retransmitir, aunque no haya un segmento sin su acuse de recibo, (S0).
 - ✓ Retransmitir sólo si hay un segmento sin su correspondiente acuse de recibo (S1).
 - ✓ No retransmitir nunca.
- **Receptor:** Por su parte el host receptor, al momento de su fallo podría estar:
 - ✓ Enviando un “acknowledgement” (A).
 - ✓ Podría estar escribiendo un segmento en memoria (W).

Crash Recovery



7 – CRASH RECOVERY (continuación).

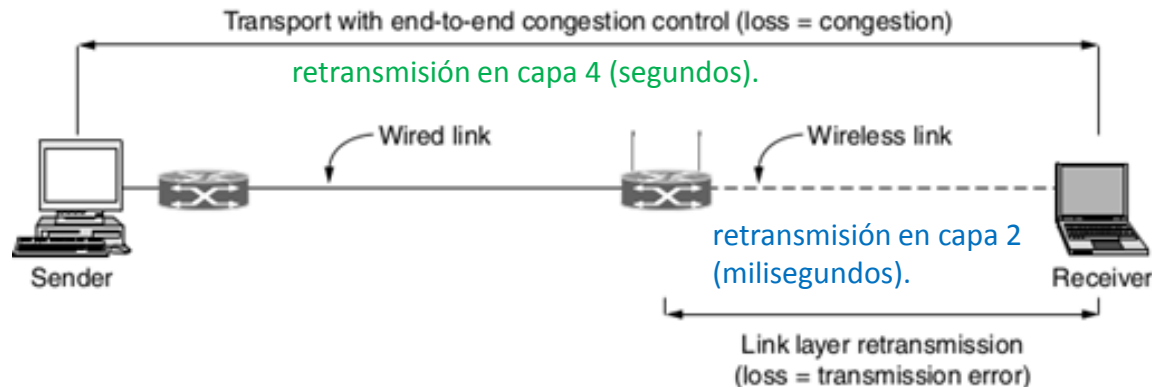
- **Estrategias:** Según como esté programado el receptor, antes escribirá (W), o antes enviará el acuse de recibo (A) y luego escribirá. Las distintas alternativas se ilustran en la figura, donde la secuencia AC(W), por ejemplo, significa que el receptor envió el “acknowledgement” primero, luego vino el “crash” y, por ende, no pudo escribir el segmento (W) correspondiente.
 - En algunos casos aparecen segmentos duplicados. En otros casos, se pierden segmentos. Y en algunas combinaciones, pueden perderse o duplicarse segmentos, como se aprecia en la figura.
- **Conclusión:** Nunca será transparente para las capas superiores, cuando se produzca una falla y un posterior restablecimiento del host. Cuánto afecte dependerá de la información que disponga la capa superior. Esta discusión puede verse en el trabajo de Saltzer et al. (1984).

Strategy used by sending host	Strategy used by receiving host					
	First ACK, then write			First write, then ACK		
	AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Always retransmit	OK	DUP	OK	OK	DUP	DUP
Never retransmit	LOST	OK	LOST	LOST	OK	OK
Retransmit in S0	OK	DUP	LOST	LOST	DUP	OK
Retransmit in S1	LOST	OK	OK	OK	OK	DUP

OK = Protocol functions correctly
DUP = Protocol generates a duplicate message
LOST = Protocol loses a message

8 – CUESTIONES ESPECÍFICAS DE REDES INALÁMBRICAS.

- Los protocolos de capa 4 deberían ser independientes de la red que subyace en capas inferiores. Sin embargo, hay algunos casos particulares:
- WiFi:** Un alto “throughput” requiere una tasa baja de pérdida de paquetes. En la práctica, una conexión “**fast TCP**” tiene una pérdida aceptable de **1%**. Sin embargo, las redes inalámbricas pierden una mayor cantidad de datos debido a su propagación por el aire, siendo normal una tasa del **10%**. Con esta tasa de error, una conexión “fast TCP” dejaría de funcionar. La solución pasa por retransmitir paquetes en capa 2, de forma que sea imperceptible para la capa 4. WiFi utiliza un protocolo “stop-wait” en capa 2, retransmitiendo varias veces en caso de errores antes de notificar a las capas superiores sobre una pérdida de datos.
- Ejemplo: En la figura se observa una conexión donde un tramo está cableado y el otro tramo es inalámbrico. Esto complica el método de control de congestión. La solución pasa por la escala de tiempos. Mientras que un error y retransmisión en capa 2 ocurre en milisegundos, una retransmisión en capa 4 se dispara en alrededor de un segundo, es decir en un tiempo 1000 veces mayor.
- Redes Satelitales:** En redes satelitales el tiempo de capa 2 es similar al tiempo límite de retransmisión de capa 4, por lo tanto, en estos enlaces se emplea corrección de errores FEC con códigos correctores, o bien no se retransmite en capa de transporte.



9 – BIBLIOGRAFÍA.

- Tanenbaum, Andrew S. y Wetherall, David J: “Computer Networks”. 5th edition. Pearson Education inc.